Propagation Network Inference from Email Communication Data

SAKIB DEWAN PRANTO
ID: 1000054037

April 13, 2026

Project Repository: `https://github.com/Glitch1258/MyCode-networkInference.git`

1 Idea

This project applies propagation network inference to temporal email communication data from a European research institution.

The core idea is to reconstruct the underlying influence graph that best explains observed email cascades, where an email from one person to another represents potential information propagation.

Using the NETINF algorithm, we infer which communication links are most likely responsible for the temporal patterns of email exchanges, treating each directed edge as a potential transmission pathway with exponentially decaying probability over time.

This method respects the temporal ordering of events: a person can only be influenced by emails they received earlier. The result is a sparse directed graph that reveals the hidden propagation structure of organizational communication.

2 Abstract

Understanding how information propagates through organizational communication networks is critical for identifying influential individuals, bottleneck structures, and potential intervention points.

This project infers a propagation network from timestamped email logs using the NETINF algorithm, which models transmission probabilities as exponentially decaying with time and greedily adds edges to maximize cascade likelihood. We group email transactions into time-window cascades (7-day windows), then construct the most likely propagation tree for each cascade by assigning every node the best possible parent among earlier activations.

Starting from an empty graph, we iteratively add the directed edge that most improves total log-likelihood across all cascades until no significant gain remains.

On the email-Eu-core-Department1 dataset (89 nodes and approximately 12,000 directed edges), we achieve a precision of 0.5. These results suggest that cascade-based propagation inference can reveal meaningful influence structure from sparse communication logs without requiring exogenous information about network topology.

3 Background Study

Understanding how information, influence, or disease spreads through networks is a key problem across biology, sociology, and computer science. Probabilistic graphical models offer a natural framework, but in many real systems—like gene regulatory networks or organizational communication—the underlying network is unobserved, and only indirect data (for example, gene expression measurements or email timestamps) are available. This creates a need for methods that infer latent network structure from observed data.

Early methods assumed static graphs, which are effective when systems are stationary. Yet many biological and social networks are dynamic. To address this, Song et al. introduced time-varying dynamic Bayesian networks (TV-DBNs), using a kernel reweighting scheme to prioritize temporally close observations together with Lasso regularization for sparsity. They validated their approach on yeast cell-cycle and EEG data.

Kolar et al. extended this line of work to undirected graphical models such as the Ising model, proposing kernel smoothing for gradual changes and total-variation penalization for abrupt shifts. Both approaches use node-wise neighborhood selection with convex optimization, and the authors provide consistency results under sparsity and smoothness assumptions.

Together, these papers lay the foundation for time-varying network inference, showing that borrowing strength across time enables dynamic structure recovery where static methods fail. The present project applies these principles to email data using a cascade-based propagation model from the influence diffusion literature.

4 Dataset

The dataset used in this project is the email-Eu-core temporal network from the Stanford Network Analysis Project. It contains anonymized email communication data from a large European research institution, including incoming and outgoing emails among institution members over approximately 800 days. The data format is a simple three-column edge list containing a source node, a target node, and a timestamp measured in Unix seconds. A separate directed edge is recorded for each recipient of each email.

For computational tractability, we use the Department 1 subnetwork, which contains communications between members of a single department. This subnetwork includes approximately 89 nodes and 12,000 timestamped directed edges. The reduced scale enables rapid experimentation while preserving the temporal structure and community dynamics of the full network.

Unlike synthetic data, where the true network is known exactly, real email data presents a ground-

truth challenge. For evaluation, we treat observed email edges as true communication events, acknowledging that the absence of an email does not necessarily indicate the absence of influence. Nonetheless, this provides a reasonable benchmark for comparing inferred influence against recorded communication.

It should be noted that we used only the first 5,000 samples out of the possible 60,000 samples. This restriction was necessary to ensure that the program could run within a reasonable time on a laptop with limited computing capability.

5 Methodology

The methodology implements the NETINF algorithm for inferring propagation networks from timestamped cascade data.

Given observed email communications grouped into temporal cascades, the algorithm reconstructs the most likely propagation pathways by assuming information spreads from earlier senders to later recipients with exponentially decaying probability over time.

The core algorithm is NETINF, which operates in two stages.

Stage 1: Most Likely Tree Construction. For each cascade, a time-ordered sequence of node activations, we compute the most likely propagation tree by assigning each node to the best possible parent among all earlier nodes in that cascade.

The transmission probability from parent u to child v given time difference dt follows an exponential distribution. For edges already in the inferred graph, the probability is rate $\times\, e^{-\text{rate}\times dt}$. For edges not yet in the graph, a small baseline probability $\varepsilon \times e^{-dt/\tau}$ ensures that unlikely transmissions can still occur, preventing zero-probability events from blocking inference.

For each node in temporal order, excluding the first node, we evaluate all candidate parents, compute the appropriate transmission probability, and select the parent with the highest probability. This produces one directed tree per cascade along with its associated log-likelihood.

Stage 2: Greedy Global Edge Addition. Starting from an empty edge set, we iteratively add edges that maximize total log-likelihood across all cascades. At each iteration, we compute the current total log-likelihood by summing the log-likelihoods of all cascade trees.

Then, for each candidate directed edge not already in the graph, we temporarily add it, recompute all cascade trees because parent assignments may change, compute the new log-likelihood and the gain, and then revert the edge. After evaluating all candidates, we permanently add the candidate with the largest positive gain. The algorithm terminates when no gain exceeds a small threshold (10^{-6}) or when a maximum number of edges (50) is reached.

Reasoning. The exponential transmission model captures the intuition that influence decays over time; an email reply is more likely minutes after the original message than weeks later. The baseline probability for non-edges serves two purposes: it allows the model to explain observations without requiring an edge, which avoids overfitting, and it provides a reference against which edge additions are compared. Only edges that substantially improve likelihood beyond this baseline are added.

This greedy approach is grounded in submodular optimization, meaning early edge additions yield the largest improvements. NETINF also models external influence through ε-edges.

The cascade grouping approach uses fixed 60-second windows to transform raw email edges into activation sequences suitable for the algorithm. Within each window, we keep only the earliest timestamp per node, ensuring that each cascade represents a temporally compact set of communications where propagation is plausible.

Background Context. The NETINF algorithm draws on ideas from the time-varying network inference literature. While Song et al. (2009) and Kolar et al. (2010) focus on kernel-weighted neighborhood selection for smoothly evolving networks, NETINF takes a different approach: it treats each cascade as an independent propagation event and greedily builds a single static graph that best explains all cascades collectively. This is appropriate for email data where communication patterns may be relatively stable over the observation period, and where the primary goal is identifying persistent influence pathways rather than tracking moment-to-moment structural changes.

6 Implementation

The implementation consists of four core modules in `netInf.py`.

Data Preprocessing (`load_email_data`, `group_into_cascades`). The email dataset is loaded from a three-column text file containing source node, target node, and Unix timestamp. Raw edges are grouped into cascades using fixed-size time windows (default 7 days = 604,800 seconds). Within each window, we extract all nodes that participated in any email and keep only their earliest timestamp, producing a temporally ordered activation sequence per cascade. This conversion from raw edges to cascades is critical because NETINF expects activation sequences rather than edge lists.

Transmission Model (`transmission_prob`). This module implements the exponential transmission probability density function. For a given time difference dt, the probability for an existing edge is rate $\times\, e^{-\text{rate}\times dt}$. For non-edges, a modified probability $\varepsilon \times e^{-dt/1000000}$ provides a small but non-zero baseline that decays slowly with time, ensuring that temporally distant transmissions receive near-zero but non-zero weight.

Most Likely Tree Construction (`most_likely_tree`). For a single cascade, the procedure iterates through nodes in temporal order, skipping the first node. For each node, it evaluates all earlier nodes as candidate parents, computes the transmission probability using either the edge-specific or baseline probability depending on whether the candidate edge already exists in the inferred graph, and chooses the candidate with the highest probability as the node's parent. The function returns the parent-child edge list and cumulative log-likelihood.

Greedy Edge Addition. The main inference loop maintains a list of inferred edges. At each iteration, it computes the current total log-likelihood over all cascades. Then it tests candidate directed edges between the first 50 nodes as a performance optimization for the email dataset. For each candidate edge, it temporarily adds the edge, recomputes total log-likelihood, calculates the gain, and then reverts the change.

After evaluating all candidates, the algorithm permanently adds the edge with the largest positive

gain. It terminates when no gain exceeds 10^{-6} or after `max_edges` iterations.

Evaluation (`compute_metrics`). The evaluation compares inferred directed edges with ground-truth email edges by converting both to undirected sets, sorting each node pair. It computes precision, recall, and F1 score against the ground-truth email transactions as a proxy for true communication relationships.

7 Results

```
Iteration 15: trying candidate edges...
  Tested 1966 edges, best gain: 0.6052
Iteration 15: added (28, 26) with gain 0.6052
Iteration 16: trying candidate edges...
  Tested 1965 edges, best gain: 0.0000
No improving edge found. Stopping.

Inferred 15 directed edges

Results (comparing inferred to email edges):
Precision: 0.733
Recall: 0.050
F1 Score: 0.094

Top 10 inferred edges:
  1. 9 -> 18
  2. 256 -> 180
  3. 256 -> 53
  4. 256 -> 81
  5. 4 -> 1
  6. 4 -> 28
  7. 12 -> 176
  8. 12 -> 223
  9. 280 -> 152
  10. 280 -> 26

Press ENTER or type command to continue
```

8 Analysis

The result indicates that the model is relatively accurate at identifying edges that influence a given node, but it still lacks the ability to adequately identify all of the influencing edges for that node.

9 Usefulness

This project demonstrates several practical utilities.

Influence Identification. One can identify which individuals serve as information hubs, namely nodes with high out-degree in the inferred propagation graph. These individuals may be critical for disseminating announcements or, conversely, may represent single points of failure in communication networks.

Bottleneck Detection. Nodes that frequently appear as parents but have few children may represent bottlenecks where information enters a team but fails to propagate further. Targeted interventions could improve information flow from these nodes.

Anomaly Detection. Sudden changes in inferred propagation structure may indicate security incidents, such as compromised email accounts sending unusual traffic, or operational disruptions such as staff departures.

10 Limitations and Future Work

Several limitations affect the current implementation.

First, the evaluation uses observed email edges as a proxy for the true influence network, which may underestimate latent influence relationships that were not directly recorded in the logs.

Second, only the first 5,000 samples were used for computational reasons, which limits coverage of the full temporal span of the dataset.

Third, the candidate edge search was restricted to the first 50 nodes as a performance optimization. While practical, this may omit valid influence relationships involving nodes outside that subset.

Future work could improve scalability, evaluate on larger portions of the dataset, and compare NETINF against alternative dynamic network inference methods.

References

[1] M. Kolar, L. Song, A. Ahmed, and E. P. Xing, "Estimating time-varying networks," *Annals of Applied Statistics*, vol. 4, no. 1, pp. 94–123, 2010.

[2] L. Song, M. Kolar, and E. P. Xing, "Time-varying dynamic Bayesian networks," in *Advances in Neural Information Processing Systems*, 2009, pp. 1732–1740.